HDOC DAY – 54

ALGORITHM:

1. Start.
2. Read size.
3. Declare the variable-length array arr[size].
4. Read the elements of arr[].
5. Read k.
6. Pass the address of arr[], size, and k to max_subarray_sum().
7. Inside the function:
    - If k <= 0 or k >= size, display -1 and return.
    - Calculate the sum of the first k elements.
    - Store this sum in maxSum.
    - Move through the remaining elements using a sliding window.
    - Subtract the element leaving the window.
    - Add the element entering the window.
    - If the new sum is greater than maxSum, update maxSum.
8. Display the maximum subarray sum.
9. Stop

C PROGRAM:

```
#include <stdio.h>


void max_subarray_sum(int *arr, int size, int k)
{
   if (k <= 0 || k >= size)
   {
      printf("-1\n");
      return;
```

```
    }

    int maxSum, sum = 0;

    // Calculate sum of first subarray of size k
    for (int i = 0; i < k; i++)
    {
        sum = sum + *(arr + i);
    }

    maxSum = sum;

    // Calculate sums of remaining subarrays
    for (int i = k; i < size; i++)
    {
        sum = sum - *(arr + i - k);
        sum = sum + *(arr + i);

        if (sum > maxSum)
        {
            maxSum = sum;
        }
    }

    printf("Maximum subarray sum: %d\n", maxSum);
}

int main()
```

```
{
    int size, k;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter %d elements: ", size);
    for (int i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    // Passing array using its address
    max_subarray_sum(arr, size, k);

    return 0;
}
```

TEST CASE TABLE:

Test	Array	k	Expected Output
1	2 1 5 1 3 2	3	Maximum subarray sum: 9
2	2 3 4 1 5	2	Maximum subarray sum: 7
3	1 2 3	4	-1

4	-2 -3 -1 -4	2	Maximum subarray sum: -4
5	5 2 8 1 6	3	Maximum subarray sum: 15